Web Application Security Assessment Report

Penetration Test – OWASP Juice Shop (Local Lab)

Field	Detail
Target	OWASP Juice Shop, http://localhost:3000
Assessment type	Black-box web application penetration test
Tester	Srimanoj R
Test period	[21/06/26] to [25/06/26]
Report version	1.0 Final
Classification	Practice Report – Training Lab

1. Executive Summary

A security assessment of the OWASP Juice Shop web application was performed to identify exploitable weaknesses, assess business impact and recommend fixes. Testing followed a 10-module methodology covering reconnaissance, traffic interception, passive scanning and manual testing of the main OWASP Top 10 vulnerability classes.

The assessment identified **[N] findings**: **[X] Critical, [X] High, [X] Medium, [X] Low**. The most serious issues let an unauthenticated attacker bypass login, take over the administrator account, read other users' data and run script in other users' browsers.

Overall risk rating: CRITICAL

Root causes are missing server-side input handling, weak authentication controls, absent object-level authorization and exposed internal resources. Fixing the top items in the remediation roadmap removes most practical attack paths.

Note: Juice Shop is intentionally vulnerable and was tested in an isolated, authorized lab.

2. Scope and Rules of Engagement

Item	Detail
In scope	Juice Shop instance on localhost (Docker), all pages and REST API endpoints
Out of scope	Host OS, other containers, external networks, third-party services
Authorization	Self-owned lab; no third-party data accessed
Testing type	Black-box, then authenticated testing with self-registered accounts
Constraints	No denial-of-service testing; no destructive actions outside the lab

3. Methodology and Tools

Testing was aligned with the OWASP Web Security Testing Guide and the OWASP Top 10.

#	Module	Level	Objective
01	Website reconnaissance	Easy	Map pages, endpoints, technologies, exposed paths
02	Burp Suite setup and HTTP interception	Easy	Capture, inspect and modify requests and responses
03	OWASP ZAP passive scanning	Easy	Identify low-risk issues without attack traffic
04	SQL Injection	Beginner	Test input handling in database-backed functions
05	Cross-Site Scripting (XSS)	Beginner	Test output encoding and script execution
06	Broken Authentication	Beginner	Test login, passwords and session handling
07	IDOR / Broken Access Control	Intermediate	Test object-level and role-based authorization
08	Security Misconfiguration	Intermediate	Check exposed files, headers, errors, defaults
09	File upload security	Intermediate	Test type, size and content validation
10	Vulnerability report writing	Professional	Document, rate and communicate findings

Tools: Burp Suite Community Edition (interception and replay), OWASP ZAP (passive scanning), Docker Desktop (hosting Juice Shop), browser developer tools.

Severity	CVSS	Meaning
Critical	9.0–10.0	Full compromise or mass data exposure, trivial to exploit

Severity	CVSS	Meaning
High	7.0–8.9	Significant data or account compromise
Medium	4.0–6.9	Limited impact or special conditions required
Low	0.1–3.9	Minor weakness or information disclosure
Info	0	Best-practice observation

4. Summary of Findings

ID	Finding	Module	OWASP	Severity	CVSS
F-01	SQL injection in login allows authentication bypass	04	A03 Injection	Critical	9.8
F-02	Administrator account uses weak password	06	A07 Auth Failures	Critical	9.8
F-03	IDOR exposes other users' baskets	07	A01 Broken Access Control	High	8.1
F-04	DOM-based XSS in product search	05	A03 Injection	High	7.4
F-05	Sensitive directory /ftp publicly browsable	08	A05 Misconfiguration	High	7.5
F-06	File upload lacks server-side validation	09	A04 Insecure Design	Medium	6.5
F-07	Verbose error messages leak stack traces	08	A05 Misconfiguration	Medium	5.3
F-08	Missing security headers	03	A05 Misconfiguration	Low	3.7
F-09	Technology and endpoint information disclosure	01	A05 Misconfiguration	Info	0

Adjust IDs, counts and ratings to match what you confirmed during testing.

5. Detailed Findings

F-01 – SQL Injection in Login (Critical)

Module: 04 | **Endpoint:** POST /rest/user/login | **CVSS:** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

Description: The email field is concatenated into a SQL query without parameterization, so injected SQL alters the query logic.

Steps to reproduce

1. Enter a single quote in the email field and observe the database error.
2. Intercept the login request in Burp and send it to Repeater.
3. Set the email to a tautology payload such as `' OR 1=1--` with any password.
4. Observe a successful login returning a token for the first user (administrator).

Impact: An unauthenticated attacker can log in as any user, including the administrator, and may extract database contents.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Use parameterized queries or prepared statements everywhere.
- Validate input server-side with allow-lists.
- Return generic errors and run the database account with least privilege.

F-02 – Weak Administrator Credentials (Critical)

Module: 06 | **Function:** Administrator login | **CVSS:** 9.8

Description: The administrator account uses a guessable password and the login endpoint has no effective lockout or rate limiting.

Steps to reproduce

1. Identify the administrator email through reconnaissance (for example, product reviews).
2. Send the login request to Burp Repeater or Intruder.
3. Try a short list of common passwords.
4. Observe successful authentication with no throttling.

Impact: Full administrative control of the application.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Enforce a strong password policy and replace weak credentials.
- Add rate limiting, lockout and CAPTCHA after repeated failures.
- Require multi-factor authentication for administrators.

F-03 – IDOR / Broken Access Control on Baskets (High)

Module: 07 | **Endpoint:** GET /rest/basket/{id} | **CVSS:** 8.1

Description: The server returns a basket based only on the numeric ID, without checking that it belongs to the authenticated user.

Steps to reproduce

1. Log in with a self-registered account and note your basket ID in the request.
2. In Burp Repeater, change the ID to another number.
3. Observe another user's basket contents in the response.

Impact: Disclosure and possible modification of other users' data; all baskets can be enumerated.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Enforce object-level authorization by checking ownership server-side on every request.
- Use unpredictable identifiers as defense in depth only.
- Add automated access-control tests.

F-04 – DOM-Based Cross-Site Scripting in Search (High)

Module: 05 | **Function:** Product search | **CVSS:** 7.4

Description: The search term is written into the page without proper encoding, so injected markup runs as script in the victim's browser.

Steps to reproduce

1. Enter a harmless HTML test payload that triggers a JavaScript alert.
2. Observe that the payload renders and the alert fires.
3. Copy the URL; opening it in another session triggers the same behavior.

Impact: Session theft, defacement and phishing through a crafted link.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Encode output contextually and avoid unsafe DOM sinks such as innerHTML.

- Deploy a restrictive Content-Security-Policy.
- Set session cookies HttpOnly and Secure.

F-05 – Exposed Sensitive Directory /ftp (High)

Module: 08 (found in 01) | **Path:** /ftp | **CVSS:** 7.5

Description: A directory listing exposes internal files, including backups and documents, to unauthenticated users.

Steps to reproduce

1. Review robots.txt and common paths during reconnaissance.
2. Browse to /ftp and observe the listing.
3. Download an exposed file to confirm access.

Impact: Disclosure of internal documents and possibly credentials or source information.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Remove the directory from the public web root or require authentication.
- Disable directory listing.
- Review the server for other leftover backup and test files.

F-06 – Insecure File Upload (Medium)

Module: 09 | **Function:** Complaint upload form | **CVSS:** 6.5

Description: File type and size restrictions are enforced mainly in the browser; intercepting the request lets disallowed types or oversized content through.

Steps to reproduce

1. Upload a permitted file and capture the request in Burp.
2. Change the extension and Content-Type to a disallowed type.
3. Forward the request and observe that the server accepts it.

Impact: Storage abuse and, in a real application, a path to stored XSS or code execution.

Evidence: [Insert screenshots / Burp request and response]

Remediation

- Validate type, extension, size and content server-side using an allow-list.
- Rename files, store outside the web root and serve with a safe Content-Type.
- Scan uploads for malware.

F-07 – Verbose Error Messages (Medium)

Module: 08 | **CVSS:** 5.3. Malformed requests return stack traces and framework details, helping attackers map the technology stack. **Remediation:** use generic error pages for users and log details server-side only.

F-08 – Missing Security Headers (Low)

Module: 03 | **CVSS:** 3.7. ZAP's passive scan reported missing or weak Content-Security-Policy, X-Frame-Options and X-Content-Type-Options, weakening defense against clickjacking, XSS and MIME sniffing. **Remediation:** add the headers at the web server or reverse proxy and re-scan.

F-09 – Information Disclosure from Reconnaissance (Informational)

Module: 01. Client-side files and responses reveal framework details, API routes and application structure. **Remediation:** minimize exposed metadata and remove unused endpoints.

6. Remediation Roadmap

Priority	Action	Findings	Target
1 – Immediate	Parameterize all queries; reset weak admin credentials; enable lockout and MFA	F-01, F-02	7 days
2 – Short term	Enforce object-level authorization; fix XSS output encoding; remove /ftp	F-03, F-04, F-05	30 days
3 – Medium term	Server-side upload validation; generic error handling	F-06, F-07	60 days
4 – Hardening	Security headers; reduce information disclosure	F-08, F-09	90 days

7. Conclusion

The application contains several critical weaknesses that can be chained: SQL injection or weak credentials lead to administrator access, while broken access control and XSS expose user data. Secure coding practices (parameterized queries, output encoding, server-side authorization) together with proper configuration management will substantially reduce risk. A retest is recommended after remediation.

Appendix A – Testing Checklist

Module	Completed	Notes
01 Reconnaissance	Yes	
02 Burp Suite interception	Yes	
03 ZAP passive scan	Yes	
04 SQL injection	Yes	
05 XSS	Yes	
06 Broken authentication	Yes	
07 IDOR / access control	Yes	
08 Security misconfiguration	Yes	
09 File upload	Yes	
10 Report writing	Yes	

Appendix B – Environment

- Juice Shop run via Docker Desktop on localhost, port 3000
- Burp Suite Community Edition proxy on 127.0.0.1:8080
- OWASP ZAP in passive scan mode
- Browser configured to proxy through Burp

Appendix C – References

- OWASP Top 10
- OWASP Web Security Testing Guide
- OWASP Juice Shop project documentation
- CVSS v3.1 specification (FIRST)

Appendix D – Penetration Test Addendum

D.1 Document Control

Field	Detail
Purpose	Practice / training exercise (no client)
Prepared by	Srimanoj R, Security Student
Reviewed by	Self-reviewed (practice)
Distribution	Personal practice – not for distribution

Version	Date	Author	Change
0.1	[Date]	[Name]	Draft
1.0	[Date]	[Name]	Final release

D.2 Testing Approach

Phase	Activities	Tools
1. Planning	Scope, rules of engagement, authorization	Written scope
2. Reconnaissance	Map pages, endpoints, technologies, hidden paths	Browser, robots.txt, dev tools
3. Scanning	Passive analysis of traffic and headers	OWASP ZAP
4. Exploitation	Manual testing of injection, XSS, auth, access control, uploads	Burp Suite Community
5. Post-exploitation	Assess impact of obtained access	Burp Suite
6. Reporting	Rate, document and recommend fixes	This report

D.3 Attack Narrative

1. **Reconnaissance** revealed the API structure, an exposed /ftp directory and the administrator's email (F-09, F-05).
2. **Initial access** was gained by bypassing login with SQL injection, or by guessing the weak admin password (F-01, F-02).
3. **Privilege escalation** occurred because the first account returned by the injected query was the administrator.
4. **Data access** followed by iterating basket IDs to read other customers' data (F-03).
5. **Reach** was demonstrated through a crafted search link delivering script to other users' browsers (F-04).

Business impact: an unauthenticated external attacker can reach full administrative control and customer data in minutes using only a browser and a free proxy tool.

D.4 Compliance Mapping

Finding	OWASP Top 10 (2021)	CWE
F-01 SQL injection	A03 Injection	CWE-89
F-02 Weak credentials	A07 Authentication Failures	CWE-521, CWE-307
F-03 IDOR	A01 Broken Access Control	CWE-639
F-04 DOM XSS	A03 Injection	CWE-79
F-05 Exposed directory	A05 Security Misconfiguration	CWE-548
F-06 Insecure upload	A04 Insecure Design	CWE-434
F-07 Verbose errors	A05 Security Misconfiguration	CWE-209
F-08 Missing headers	A05 Security Misconfiguration	CWE-693

D.5 Retest Results

ID	Finding	Original severity	Retest date	Status
F-01	SQL injection	Critical	[Date]	[Fixed / Open]
F-02	Weak credentials	Critical	[Date]	[Fixed / Open]
F-03	IDOR	High	[Date]	[Fixed / Open]
F-04	DOM XSS	High	[Date]	[Fixed / Open]
F-05	Exposed /ftp	High	[Date]	[Fixed / Open]
F-06	Insecure upload	Medium	[Date]	[Fixed / Open]

D.6 Limitations and Disclaimer

This assessment reflects the application at the time of testing and covers only in-scope assets. Testing was time-boxed and used Community editions of the tools, so absence of a finding does not guarantee absence of vulnerabilities. All testing was performed in an authorized, isolated lab and must not be repeated against systems without written permission.

D.7 Glossary

Term	Meaning
IDOR	Insecure Direct Object Reference: accessing objects by changing an identifier
XSS	Cross-Site Scripting: injecting script into pages viewed by others
SQLi	SQL Injection: altering database queries through user input
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
PoC	Proof of Concept